

Lumen Ops Build Week

Fifteen exercises that build a municipal street-lighting operations console from scratch, on the exact stack and the exact patterns used in Front App Espacios Verdes — so that on Monday week, that codebase reads as familiar rather than foreign.

WHAT EXISTS BY FRIDAY EVENING

- ▶ A standalone Angular 19 app, no NgModules
- ▶ JWT auth, three interceptors, an app initializer
- ▶ A filter DSL and a reusable paginated table base
- ▶ Reactive forms with a custom control and a dirty-exit guard
- ▶ An OpenLayers map wired to the table
- ▶ A Karma suite, tuned budgets, a Dockerfile and CI
- ▶ A mock API with the real pagination envelope
- ▶ Route guards and role-aware structural directives
- ▶ The same base rewritten with signals and `resource()`
- ▶ Transloco in two locales, loaded per feature
- ▶ ECharts time series on a dashboard

Contents

DAY 01 Skeleton, contract, shell

- 1.1 Scaffold a workspace that matches
- 1.2 A mock API that speaks the real dialect
- 1.3 Application shell and lazy routes

DAY 02 Who you are and what you may do

- 2.1 Session, token, and a bootstrap that waits
- 2.2 Guards, and a home page that depends on who you are
- 2.3 Role-aware structural directives

DAY 03 The data layer and the table pattern

- 3.1 A typed API service and errors that say something
- 3.2 A filter DSL, not a bag of query params
- 3.3 The abstract paginated table, RxJS edition

DAY 04 Signals, forms, and everyone else

- 4.1 The same base class, rebuilt on signals
- 4.2 Forms that refuse to lose work
- 4.3 Two locales and a keyboard

DAY 05 Map, charts, and shipping it

- 5.1 An OpenLayers map wired to the table
- 5.2 Energy time series
- 5.3 Make it defensible

Ground rules

You are building **Lumen Ops**: a console for a city's street-lighting contract. Luminaires are geolocated assets with a lamp type and a wattage. Citizens and council inspectors report faults against them. Faults get validated, turned into work orders, and assigned to contractor crews. Each luminaire reports energy consumption over time. Two organisations share the app — the council and the maintenance contractor — and they must not see the same buttons.

WHY STREET LIGHTING AND NOT GREEN SPACES

Every structure in that list has a twin in the real codebase: faults are *incidencias*, crews are *grupos de trabajo*, work orders are *órdenes de trabajo*, energy readings are *series temporales*, council and contractor are `AYTO` and `EMPRESA`.

Different nouns, identical architecture. That gap is deliberate: if the domain matched you would pattern-match on names, and the day you open an unfamiliar module you would be lost again. Here you have to recognise the *shape*.

MATCH THE REAL REPO'S CONVENTIONS EXACTLY

The point of a training repo is that your eye adapts. Copy these settings verbatim, even where you disagree with them — day 5 is when you get to argue.

SETTING	VALUE	WHY IT IS LIKE THAT
Angular	19.2	Pin it. Control flow, <code>resource()</code> and signal inputs all behave differently across majors.
Components	standalone, SCSS	The real app has 4 NgModules left out of ~155 components. Write zero.
Prettier	tabs · single quotes · 100 cols · no trailing comma	Unusual combination. Adopting it now means real diffs look normal later.
tsconfig	strict: false · strictTemplates: true	Deliberately the real setting. Loose TypeScript, strict templates. You will feel both.
Aliases	@core/* @shared/* @business/*	Same alias names, so imports read the same.
Router	hash location · component input binding · preload all	All three are non-default choices in the real app. You need to know what each costs.
UI base	Bootstrap 5 + SCSS	No Tailwind, no Material. A variables/mixins/variants layer you write yourself.

DO NOT INSTALL THE @APPOLOW PACKAGES

The real app leans on seven private `@appoLow/*` packages for its table, forms, modals and kanban. You are going to build simplified versions of the two that matter most — the paginated table and the modal service — by hand.

This is the highest-value decision in the whole week. Someone who has written their own paginated table base class reads `@appoLow/paginable` as a set of choices they recognise. Someone who has only consumed it reads it as magic, and stays stuck the first time it behaves unexpectedly.

WORKING METHOD

1. One branch and one commit per exercise: `drill/1-2-mock-api` . Conventional commit messages.
2. Each exercise ends with the app running. Never leave the repo broken overnight.
3. Write the notes each exercise asks for into `docs/` . The written answers are half the learning and all of the evidence.
4. Budget 6–7 hours a day. Exercises tagged `CORE` carry the day; the rest can slip.

Nothing domain-specific today. You are laying the foundation that every later exercise stands on: a workspace configured like the target, a fake backend that speaks the real API's dialect, and an application shell with lazy routes.

1.1 Scaffold a workspace that matches

CORE

2H

TOOLING

A default `ng new` gives you something subtly different from the target in six ways. Close every gap deliberately and write down what each one does.

DO

1. Scaffold with `npx @angular/cli@19 new lumen-ops --style=scss --ssr=false --routing`, then delete anything the app does not need.
2. Write `.prettierrc` to the spec in the table above, plus an `.editorconfig` that agrees with it. Verify by formatting a file and checking the diff is tabs.
3. Add the path aliases to `tsconfig.json`. Set `strict: false` and, under `angularCompilerOptions`, `strictTemplates: true`. Then write two paragraphs in `docs/tsconfig-notes.md` on what that combination actually permits and forbids — this catches people out constantly.
4. Install Bootstrap 5 and wire it up: SCSS entry in `angular.json` styles, the JS bundle in scripts. Create `src/styles/` with `_variables.scss`, `_buttons.scss`, `_tables.scss` and a `components/variants.scss` holding a mixin that generates status colour variants from a Sass map.
5. Configure the router in `app.config.ts` with `withHashLocation()`, `withComponentInputBinding()` and `withPreloading(PreloadAllModules)`. For each, write one sentence on what breaks without it and what it costs.
6. Set production budgets deliberately low to begin with — 1 MB initial warning. You want the build to complain as the app grows, so you notice.

DONE WHEN

- `ng build --configuration=production` and `ng test` both pass on the empty app.
- Your variant mixin generates at least four status colours from one Sass map, with no repeated hex values.
- `docs/router-options.md` answers all three router questions in your own words.

IN THE REAL REPO

`angular.json`, `tsconfig.json`, `.prettierrc`, `src/styles/` and `src/app/app.config.ts`. Note the real budget is 4 MB — you will find out on day 5 why yours should not be.

1.2 A mock API that speaks the real dialect

CORE2.5HAPI CONTRACT

This is not throwaway scaffolding. The pagination envelope and the filter payload you implement here are the contract every later exercise consumes, and they are copied from the real backend's shape — including the parts that look odd.

THE ENVELOPE YOU MUST RETURN

```
// POST /luminaires/paged
// request
{
  page: 1,                // one-based
  perPage: 20,
  searchTerm: 'calle mayor',
  searchKeys: ['code', 'street'],
  ordination: { property: 'code', direction: 'ASC' },
  filters: []              // the DSL from exercise 3.2
}

// response
{
  data: [ ... ],
  currentPage: 1,
  lastPage: 12,
  total: 231,
  perPage: 20
}
```

DO

1. Build `mock-api/` as a small Express server. `json-server` underneath is fine, but the paged endpoints need custom middleware — its default pagination does not produce this envelope.
2. Seed roughly 600 luminaires across a real bounding box for a city you pick, each with `code`, `street`, `lat`, `lon`, `lampType` (SODIUM · LED · METAL_HALIDE), `wattage`, `installedAt`, `status` and `zoneId`. Then ~400 faults, ~250 work orders, 12 crews.
3. Implement `POST /auth/login` returning a JWT-shaped token whose base64 payload carries `sub`, `roles` and `exp`. It does not need a real signature — it needs the right shape, because your interceptor will decode it on day 2. Seed four users: ADMIN, COUNCIL, CONTRACTOR, VIEWER.
4. Add `GET /luminaires/:id/energy?from=&to=` returning hourly kWh readings. You need real time series on day 5, so generate something with a visible daily cycle rather than noise.
5. **Add artificial latency** — 300–900 ms, randomised, on the paged endpoints. Without it, debouncing and race conditions are invisible on localhost and two later exercises become meaningless.
6. Add two deliberate traps you will need later: an id that always returns 500, and an endpoint that takes 4 seconds.

7. Configure `proxy.conf.json` so the Angular dev server forwards `/api` to the mock. Do not hardcode a port in the app.

DONE WHEN

- Page 1 and page 12 return different data with a correct `lastPage`.
- Sorting by `code DESC` genuinely reverses the result.
- Login with each of the four users returns a decodable token with the right roles.
- The whole thing starts with one command alongside `ng serve`.

IN THE REAL REPO

`src/app/shared/models/paginated-data.ts` defines exactly this envelope;
`src/app/core/auth/interfaces/paged-data-request-param.ts` defines the request. Note the real app carries *two* overlapping param interfaces — `QueryParams` and `PagedDataRequestParam` — which is the kind of drift you will now be able to spot and name.

1.3 Application shell and lazy routes

2H

ROUTING

Every page in the target renders inside one persistent wrapper with a collapsible sidebar, a header and breadcrumbs. Build that, and load every feature lazily from the first commit rather than retrofitting it.

DO

1. Create `layout/` with `wrapper`, `aside`, `header`, `footer` and `logo` components. The wrapper hosts a `router-outlet`; the aside collapses and remembers its state.
2. Set up the route tree: a root route rendering the wrapper with children, and an `/auth` branch outside it. Everything under the wrapper uses `LoadComponent`.
3. Create four feature routes as placeholders: `luminarias`, `averias`, `ordenes-trabajo`, `cuadrillas`. Yes, Spanish route segments — the real app mixes English code with Spanish routes and you should experience that rather than be surprised by it.
4. Build breadcrumbs driven by route `data.breadcrumb`, walking the activated-route tree. Handle a child route with an `:id` segment resolving to a real label.
5. Confirm in the network tab that navigating to a feature loads a chunk that was not in the initial bundle.

DONE WHEN

- Four routes load as separate chunks and the shell does not remount between them.
- Sidebar collapse survives a page reload.
- Breadcrumbs render correctly three levels deep.

IN THE REAL REPO

`src/app/layout/`, `src/app/app.routes.ts` and the 972-line `src/app/business.routes.ts`. Once you have built your own you can read that file as data rather than as a wall.

Authorisation in this kind of app is not one guard on one route. It is a resolved user before bootstrap completes, guards that decide which home page you even have, and directives that remove buttons from the DOM. Build all three layers and understand why there are three.

2.1 Session, token, and a bootstrap that waits

CORE

2.5H

AUTH

The application must know who the user is before the first route resolves. Angular gives you `provideAppInitializer` for that, and getting it wrong produces a class of bug that is very hard to read backwards from the symptom.

DO

1. Build `core/auth/` with an `AuthService` holding the current user in a signal, plus `login`, `logout` and `initializeUser()`.
2. Decode the JWT payload to read roles and expiry. Write your own base64url decode rather than installing a library — it is eight lines and you should know what is inside a token.
3. Register `initializeUser()` through `provideAppInitializer`. Then deliberately make it reject and observe what the user sees. Fix it so a failed session resolution lands on the login page instead of a blank screen, and write down what you saw before the fix.
4. Write a JWT interceptor that attaches the bearer token. Decide explicitly whether it should skip the login endpoint and whether it should attach to absolute URLs pointing elsewhere — both matter.
5. Write an `AcceptLanguage` interceptor that sets the header from the current language. Keep language state in one service; do not let components reach into `localStorage` directly.
6. Build the login page and a logout route.

DONE WHEN

- Reloading on a deep route keeps you logged in and does not flash the login screen.
- An expired token sends you to login rather than producing a wall of 401s.
- No component imports `localStorage`.
- `docs/app-initializer.md` describes the failure mode you produced on purpose.

IN THE REAL REPO

`src/app/core/auth/services/`, the two `provideAppInitializer` blocks in `app.config.ts`, and `core/auth/interceptors/jwt.interceptor.ts`. The real app reads `localStorage` directly in 11 files — after this exercise you will have an opinion about that.

2.2 Guards, and a home page that depends on who you are

CORE

2.5H

GUARDS

A contractor has no business on the council's fault-validation screen, so `/` cannot mean the same thing for everyone. The real app solves this with several competing empty-path redirects, each behind a `canMatch`. It is an unusual pattern and worth building once.

DO

1. Write three functional guards: `authGuard` (is there a session), `rolesGuard` (has *all* listed roles), `someRoleGuard` (has *at least one*). Read permissions from route `data`.
2. Define `ROLE_GROUPS` as a frozen constant — `ALL`, `COUNCIL`, `CONTRACTOR`, `ADMIN` — and have routes reference groups, never role strings inline.
3. Now the interesting part. Declare four empty-path redirects at the root, each with a different `canMatch`, so that a council user lands on `averias`, a contractor on `ordenes-trabajo`, a viewer on `luminarias`, and anyone else on a fallback.
4. Establish by experiment what the difference between `canMatch` and `canActivate` is here, and why only one of them can drive this pattern. Write the answer down — this is the single most useful thing you will learn today.
5. Add an `/unauthorized` page and route denied navigations there rather than silently failing.
6. Write specs for all three guards, using a fake `AuthService`.

DONE WHEN

- Logging in as each of the four users lands on a different home page.
- Typing a forbidden URL directly redirects to `/unauthorized`.
- Guard specs cover allow, deny and no-session for each guard.
- `docs/canmatch-vs-canactivate.md` explains the difference with your own example.

IN THE REAL REPO

`src/app/core/guards/homepage.guard.ts` and the six-way redirect at the top of `src/app/app.routes.ts`, with its comment listing the evaluation order.

2.3 Role-aware structural directives

2H

DIRECTIVES

Guards protect routes. They do nothing about a delete button sitting inside a table row. The target repo has six structural directives for this, in near-identical pairs, and reading them is confusing until you have written them.

DO

1. Write `*lumenWithRoles` (render only with all listed roles) and `*lumenWithSomeRole` (render with any), using `TemplateRef` and `ViewContainerRef`.
2. Write the inverted pair, `*lumenHideWithRoles` and `*lumenHideWithSomeRole`. Before writing them, argue in your notes whether they earn their place or whether a negation flag would be better. The real repo chose four directives; decide whether you agree.
3. Make them react to the user signal, so a role change re-evaluates without a reload.
4. Add an `abilities` object on one feature component — a flat record of booleans computed once from the current user — and use it for logic that is not simply show/hide.
5. Apply all of it to a fault list: VIEWER sees rows only, COUNCIL can create and validate, CONTRACTOR can close, ADMIN can delete.
6. Write a spec asserting that a VIEWER's rendered DOM contains no write actions. This is a security-shaped test and the most valuable one in the exercise.

DONE WHEN

- The same page renders four genuinely different action sets for the four users.
- Hidden actions are absent from the DOM, not merely invisible — verify in the inspector.
- Your notes take a position on whether four directives were the right call.

IN THE REAL REPO

`src/app/core/auth/directives/` — all six — and the `abilities` object in `business/widgets/incidencias-pendientes-61051-table/`.

Today is the centre of the week. Almost every screen in an application like this is a filtered, sorted, paginated table over a REST collection, and the whole target codebase is organised around one abstract base class that solves it. You are going to write that class.

3.1 A typed API service and errors that say something

2H

HTTP

One service wraps `HttpClient` for the whole app. Type it properly from the start — the real one returns `Observable<any>` everywhere and you will see exactly what that costs downstream.

DO

1. Write `ApiService` with generic methods — `get<T>(path, params?): Observable<T>` — defaulting to `unknown`, never `any`. Include blob variants; you need them for exports on day 5.
2. Build query params with `HttpParams`. Note it is **immutable**: `params.set()` returns a new instance and mutating one does nothing. Write a spec that would fail if someone reached for `Object.assign` instead — because in the real repo, someone did, at three call sites.
3. Write an error interceptor that maps status codes to translated, actionable messages, and surfaces them through a toast service. "Error 500" tells the user neither what happened nor what to do.
4. Add bounded retry with backoff for idempotent requests *only*. A retried POST that files a duplicate fault is worse than the original error; make that structurally impossible, and prove it with a spec.
5. Make sure a 401 still reaches the auth flow rather than being swallowed by your handler.
6. Test against the two traps you planted in the mock API: the always-500 id and the 4-second endpoint.

DONE WHEN

- No method on `ApiService` returns `any`.
- A spec proves retry never fires on POST, PUT or DELETE.
- DevTools offline mode produces one clear toast, not a cascade.

IN THE REAL REPO

`src/app/core/api/services/api.service.ts` and `core/api/interceptors/api-error.interceptor.ts`. Compare the `HttpParams` handling around line 27 with what you just wrote.

3.2 A filter DSL, not a bag of query params

CORE

2H

DATA MODELLING

When filters must be composable, come from four different sources, and serialise to a backend that understands operators, `?status=OPEN&severity=HIGH` stops being enough. The target app models each condition as a structured object, and this is the single hardest thing to read in that codebase cold.

THE SHAPE TO IMPLEMENT

```
type Filters = Array<Filter | Array<Filter>>; // nesting expresses precedence

interface Filter {
  leftHand: { type: AssignmentType; value: string | number };
  matchMode: MatchMode; // EQUAL, CONTAINS, GT, IN, IS_NULL...
  rightHand?: { type: AssignmentType; value: string | number | Array<...> };
  operator?: 'AND' | 'OR'; // chains to the next condition
}
```

DO

1. Implement the types above, with `AssignmentType` covering at least `STATIC`, `CONTROL` and `QUERY_PARAM`, and match modes split by operand type — string, number, null.
2. Write `buildFilterConditions(record)`, turning a flat object of form values into `Filters`, choosing the match mode from the value's type: a string becomes `CONTAINS`, an array becomes `IN`, a null becomes `IS_NULL`.
3. Write `flattenFilters()` for the nested case, and make your mock API actually evaluate the DSL — including `OR` chains and one level of nesting. Implementing the evaluator is what makes the format make sense.
4. Cover it with a real spec suite. Every table in the app will funnel through this function, and a wrong condition shows a user the wrong faults rather than an error — silent and worse.
5. Write `docs/filter-dsl.md` explaining the format to a newcomer in under a page.

DONE WHEN

- A form producing `{ status: 'OPEN', severity: ['HIGH', 'MEDIUM'], street: 'mayor' }` yields correct conditions and correct results.
- An `OR` group nested inside an `AND` chain evaluates correctly end to end.
- Empty, null and empty-array values produce no condition at all rather than a condition matching nothing.

IN THE REAL REPO

`src/app/shared/models/filter.ts`, `src/app/shared/utils/filters.ts` and `src/app/core/types/operators.ts`. This exercise exists specifically so those three files are readable on first contact.

3.3 The abstract paginated table, RxJS edition

CORE3HRXJS

Write the base class that fifteen screens will extend. Search, sort, paginate, filter, refresh, delete-with-confirm — once, generically, with the async coordination handled properly.

DO

1. Define `GenericCollectionService<T>`: the interface a base class needs from any collection — `paged()`, `get()`, `save()`, `delete()`.
2. Write `abstract class PaginatedTableComponent<T extends { id?: string | number }>` holding params in a `BehaviorSubject`, deriving the data stream with `combineLatest` → `debounceTime` → `switchMap`.
3. `switchMap` is the load-bearing choice. Prove it: temporarily swap it for `mergeMap`, type fast in the search box against your artificial latency, and watch stale results win. Screenshot it for your notes, then change it back.
4. Handle loading and error as explicit state, and make sure an error on page 4 does not leave the table showing page 3's rows with no indication.
5. Build a presentational `<lumen-table>` taking a header config array and cell templates via directives, so columns are declared in the consuming component rather than hardcoded.
6. Extend it twice — a luminaires table and a faults table — and confirm each subclass adds only its own config and its own service.
7. Add delete-with-confirmation wired through a confirm dialog.

DONE WHEN

- Two tables work fully, and the second subclass is under 80 lines.
- Rapid typing never renders results for a stale query.
- Sorting a column is one request, not two.
- Your notes contain the `mergeMap` evidence.

IN THE REAL REPO

`src/app/shared/components/paginated-table/` and `src/app/shared/generic-collection.service.ts`. The presentational table is `@apollo/paginate` — now a library whose choices you can evaluate rather than guess at.

Yesterday's table was written the way the older half of the target repo is written. Today you rewrite it the way the newer half is, then build the form layer — and finish with translation and accessibility, which in a public-sector application are requirements rather than polish.

4.1 The same base class, rebuilt on signals

CORE

2.5H

SIGNALS

Keep yesterday's class. Write a second one beside it using `signal`, `computed`, `model` and `resource()`, migrate one table onto it, and hold both in your head at once. The target repo has exactly this pair, undocumented, and someone has to be able to explain the difference.

DO

1. Write `v2/PaginatedTableComponent`: params as signals, filters as a `computed` composed from several sources, and the fetch expressed with `resource()`.
2. Compose filters from four inputs the way the real widget does — component inputs, router state, post-filters and dynamic form filters — merged in one `computed`. Use `model()` for the two-way ones.
3. Answer the question that yesterday's `switchMap` experiment set up: how does `resource()` handle a slow response arriving after a newer request? Test it against your artificial latency and write down the mechanism, not just the observation.
4. Handle loading and error the `resource()` way rather than porting your boolean fields across, and note what you gained and what you lost.
5. Migrate the faults table to v2, leaving luminaires on v1. Behaviour must be indistinguishable to the user, debounce feel included.
6. Write `docs/table-v1-vs-v2.md` as a side-by-side: param changes, debouncing, loading, errors, cleanup, and which you would choose for a new screen and why.

DONE WHEN

- Both tables behave identically from the user's side while sharing no base class.
- Neither renders a stale result set under fast input.
- Your comparison doc is specific enough to hand to someone facing the same migration.

IN THE REAL REPO

`src/app/shared/v2/components/paginated-table.component.ts` versus `src/app/shared/components/paginated-table/`. Both live in the codebase today with nothing written down about when to use which — your doc would be a genuine contribution.

4.2 Forms that refuse to lose work

CORE

3H

FORMS

A fault report has a date range, a severity, a luminaire picked from 600 options, and photos. Build it once, properly, with the pieces that make form-heavy apps maintainable: validators that live outside the component, a control that is reusable, and an exit that asks first.

DO

1. Build the reactive form for creating and editing a fault. One component for both modes — work out how to make that clean rather than a chain of `if (isEdit)`.
2. Write a cross-field validator for the date range as a `FormGroup` validator in `shared/validators/`. Surface the error on the end-date control, not only on the group, or the user never sees it. Spec it with at least four cases.
3. Build a luminaire picker as a proper `ControlValueAccessor`: typeahead against the API, debounced, keyboard navigable. It must work inside `formControlName` like any native control, including `disabled` state and `touched` propagation.
4. Write a small modal service over the CDK overlay: open a component, return a promise of a result, close on backdrop and on Escape. Trap focus while open and restore it on close.
5. Write a `@Confirmable()` method decorator that shows a confirm dialog before the decorated method runs and cancels it on decline. This is a pattern the target repo leans on heavily and it is genuinely instructive to implement.
6. Add a `CanDeactivate` guard for dirty forms, using your modal rather than `window.confirm`. Reset dirty state after a successful save so saving then leaving does not prompt.

DONE WHEN

- An invalid date range blocks submit with the message on the right control.
- The picker works under `formControlName`, including disabled state.
- Leaving a dirty form prompts once; leaving a saved form does not.
- `@Confirmable()` works on at least two unrelated methods.

IN THE REAL REPO

`src/app/shared/decorators/ (confirm, check-form-validity, check-form-dirt),`
`src/app/shared/validators/`, and the 1,426-line `business/widgets/edistar-incidencia-61053-form/`
— which you will now be able to read as a large instance of a pattern you know.

4.3 Two locales and a keyboard

2H

I18N · A11Y

Translation drifts the moment one locale gets a key the other does not, and a municipal contract usually names an accessibility standard. Do both properly now, while the app is small enough that it takes two hours.

DO

1. Install Transloco with a custom HTTP loader, `es` and `en`, Spanish as default and fallback.
2. Translate every user-facing string. Hunt specifically for `placeholder`, `title` and `alt` attributes — that is where hardcoded text hides, and it is exactly where the real repo still has Spanish sitting in templates.
3. Load feature translations lazily: a `@LoadTranslations` class decorator that registers a scoped translation file when the component loads, so the root bundle does not carry every string.
4. Write `scripts/check-i18n.js`: report keys present in one locale and missing from the other, flag empty values, exit non-zero on any finding. Then delete a key on purpose and confirm it fails usefully.
5. Navigate the whole app by keyboard only. Log every place focus vanishes, gets trapped, or lands somewhere invisible.
6. Fix the shell: landmarks, `aria-expanded` on sidebar toggles reflecting real state, `aria-current` on the active route, a skip-to-content link that is visually hidden but never hidden from focus, and a focus ring that works on both your light and dark surfaces.
7. Run Lighthouse and axe DevTools on three pages. Record the numbers.

DONE WHEN

- `node scripts/check-i18n.js` exits 0, and non-zero when you break it on purpose.
- Switching language re-renders without a reload and survives one.
- The app is fully operable by keyboard with focus visible at every step.
- Lighthouse accessibility is above 95 on all three pages.

IN THE REAL REPO

`src/app/shared/decorators/load-translations.decorator.ts`, `transloco-loader.service.ts`, and the 24 per-widget `assets/i18n/` folders. For the record, its `es.json` currently carries 644 keys against 634 in `en.json` — the drift your script exists to catch.

The two heavy dependencies that define an app like this — a real GIS map and a charting library — plus the work that turns a training repo into something you would actually defend in review.

5.1 An OpenLayers map wired to the table

CORE

2.5H

GIS

600 geolocated luminaires on a slippy map, with the map and the table as two views of one selection. This is the exercise that makes the target repo's 1,400-line map component approachable.

DO

1. Add OpenLayers and build a `<lumen-map>` with an OSM base layer and a vector layer of luminaires. Style points by `status`, reusing the same SCSS variant colours as your status pills — one source of truth for meaning.
2. Cluster the points. 600 raw features is already visibly slow to interact with; find the threshold yourself rather than taking that on trust.
3. Wire selection both ways: clicking a feature highlights the table row, selecting a row pans and highlights the feature.
4. Make the map obey the active filters from exercise 3.2, so filtering the table visibly filters the map.
5. Handle the lifecycle properly. An OL map holds listeners and a resize observer; make sure destroying the component releases all of it, and verify with the memory profiler across ten navigations that nothing accumulates.
6. Add a popup on feature click with a link to the luminaire detail route.

DONE WHEN

- Map and table stay in sync in both directions.
- Panning and zooming with all 600 points stays smooth.
- Ten navigations in and out leave no detached listeners.

IN THE REAL REPO

`src/modules/elliott/components/ol-map/` and `src/app/shared/components/openlayers-detail/`. Note that the whole `elliott` folder is a git submodule — worth understanding as a structural choice.

5.2 Energy time series

1.5H

CHARTS

A dashboard over the hourly readings you seeded: consumption over time, a breakdown by lamp type, and faults by severity. The lesson here is as much about weight as about charts.

DO

1. Add ECharts via `ngx-echarts` and build a line chart of consumption with a selectable date range.
2. Add a breakdown by lamp type and a fault-severity distribution. Take the semantic colours from your SCSS variants rather than from ECharts' defaults, so severity means the same colour everywhere in the app.
3. Make charts respond to the app's language: axis labels, tooltips and number formatting all go through the locale.
4. Now the real lesson. Measure what ECharts adds to the bundle, then import only the components you use instead of the full build, and measure again. Put both numbers in your notes.
5. Handle empty ranges and single-point series without the chart collapsing or throwing.

DONE WHEN

- Three charts render real seeded data and respond to the date range.
- Switching to English relabels the axes.
- Your notes carry the before/after bundle figures for the selective import.

IN THE REAL REPO

`src/app/shared/utils/charts.ts` and the dashboard module. The real app imports ECharts wholesale — worth checking what that costs it.

5.3 Make it defensible

CORE

3H

QUALITY · DELIVERY

A week's code is not finished until someone else could run it, review it, and deploy it. This last exercise is also where you finally get to argue with the settings you copied on Monday.

DO

1. Bring the suite up: specs for the filter builder, the guards, the pipes, the cross-field validator, and one component spec proving a VIEWER's DOM contains no write actions. Enable coverage thresholds in `karma.conf.js` just above whatever you reach, so it can only go up.
2. Build with `--stats-json` and rank the ten heaviest contributors to the initial bundle. You now have OpenLayers, ECharts, Bootstrap and FontAwesome in one app — the same weight problem the real repo has.
3. Defer at least one heavy dependency behind a lazy boundary. Measure before and after. Then revisit `PreloadAllModules` from exercise 1.1 with actual numbers and decide whether to keep it.
4. Turn on `strictNullChecks`. Record how many errors appear and which files produce most of them. Fix them if you can in the time; if not, write the migration plan — that survey is the deliverable either way.
5. Write a `Dockerfile` serving the built app through nginx, a `docker-compose.yml` that brings up the app and the mock API together, and a CI pipeline that lints, tests and builds.
6. Write the `README.md` you wish the real repo had: what this is, how to run it, how the folders are organised, and the three architectural decisions a newcomer most needs explained.

DONE WHEN

- `docker compose up` serves a working app with data, from a clean clone.
- CI is green and coverage thresholds are enforced.
- The bundle is measurably smaller than before the deferral, with both numbers recorded.
- Someone who has never seen the repo can run it from the README alone.

IN THE REAL REPO

`Dockerfile`, `docker-compose.yml`, `.gitlab-ci.yml`, `karma.conf.js` — and a `README.md` that is still the GitLab default template, which tells you something about where documentation sits on most project backlogs.

Now open the real repo

Clone `front-app-aguayjardin` on the Monday after. Do not start at `src/app/business/` — 31 widgets of generated code is where people give up. Read it in the order you built yours, and every file below should land on something you already have hands-on knowledge of.

1. **app.config.ts, then app.routes.ts.** You wrote both. Find the six-way `canMatch` redirect and confirm it is the pattern from exercise 2.2.
2. **core/auth/ end to end.** Services, interceptors, six directives, three guards. Compare their four hide/show directives against the two you decided were enough.
3. **shared/models/filter.ts and shared/utils/filters.ts.** The hardest files in the codebase to read cold, and now the ones you know best.
4. **Both paginated table base classes.** `shared/components/paginated-table/` and `shared/v2/`. You have written both and documented the difference; check whether your conclusions hold.
5. **One generated widget, all the way down.** `incidencias-pendientes-61051-table`. Every import in it now resolves to a concept you have implemented — signals, the filter composition, the role directives, the `@LoadTranslations` decorator, the `abilities` object.
6. **scripts/reset-schematics.js.** The thing you could not have guessed: `src/app/business/` is generated from Appolow schematics and can be regenerated or deleted wholesale. That single fact explains most of why the codebase is shaped the way it is, and why durable logic has to live in `shared/` and `core/`.
7. **Only then, the elliot submodule.** Maps and tree views, 4,000-line components, a separate repository. Save it for week two.

THE MEASURE OF A GOOD WEEK

Not that your Lumen Ops is impressive — it is a training repo and nobody will run it. It is that on the Monday after, you can open an unfamiliar widget in a 513-file codebase, recognise the pattern it is an instance of, and change it without breaking the role gating.

Modelled on `front-app-aguayjardin` @ develop · Angular 19.2 · 15 exercises · 5 days